

簡易的網路 Promise fetch 範例

在這個範例中，我們定義了 `FetchData2` 函式組件，並將 `userID` 作為參數解構出來。在 `useEffect` 鉤子中，我們使用 `fetch` 函式來發送 GET 請求，並在 URL 中使用模板字串 `${userID}` 來傳遞 `userID` 的值。接著，我們將回應轉換為 JSON 格式的資料，並在控制台中顯示出來。

最後，我們將 `FetchData2` 元件作為子元件在父元件中使用，並將 `userID` 屬性設置為字串 "3"：

```
import React, { useEffect } from 'react';

const FetchData2 = ({ userID }) => {
  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await
fetch(`https://jsonplaceholder.typicode.com/posts/${userID}`);
        const data = await response.json();
        console.log('User Data:', data);
      } catch (error) {
        console.log('Error:', error);
      }
    };

    fetchData();
  }, [userID]);

  return <div>Fetching Data...</div>;
};

export default FetchData2;
=====
import React from 'react';
import FetchData2 from './FetchData2';

const App = () => {
  return (
    <div>
```

```
        <h1>My App</h1>
        <FetchData2 userID="3" />
    </div>
);
};

export default App;
```

請確保在實際應用中將 **URL** 替換為您需要使用的 **API** 端點，並根據需求調整資料處理的邏輯。

元件用於從 API 抓取電影資料並在畫面上顯示。

首先，在函式元件中使用了 `useState` 鉤子來建立一個狀態 `movieData`，並初始化為 `null`。這個狀態用於存儲從 API 抓取的電影資料。

接下來，我們使用 `useEffect` 鉤子，它用於在組件渲染完成後執行副作用。在這個範例中，副作用是進行資料抓取。

在 `useEffect` 內部，我們定義了一個名為 `fetchData` 的 `async` 函式，用於發送 HTTP GET 請求並獲取電影資料。我們使用 `fetch` 函式發送請求到指定的 API 網址。

當請求成功返回後，我們使用 `response.json()` 方法將回傳的資料解析為 JSON 格式。然後，我們使用 `console.log` 將資料輸出到控制台，以便檢查是否成功獲取資料。接著，我們使用 `setMovieData` 函式將資料設定到 `movieData` 狀態中。

如果在請求過程中發生錯誤，我們使用 `catch` 區塊捕獲錯誤，並使用 `console.log` 將錯誤輸出到控制台。

接著，我們使用條件渲染來檢查 `movieData` 是否為 `null`。如果是 `null`，表示資料還在抓取中，我們顯示一個簡單的文字訊息「Fetching Data...」。

最後，在 JSX 部分，我們使用 `movieData` 狀態來顯示電影資料。在這個範例中，我們假設電影資料是一個陣列，並從 `movieData[0]` 中顯示第一個電影的標題和描述。您可以根據實際的資料結構做相應的修改，以顯示更多電影資料。

```
import React, { useEffect, useState } from 'react';
```

```
const FetchData3 = () => {
```

```
  const [movieData, setMovieData] = useState(null);
```

```
  // 新增狀態 movieData，初始值為 null
```

```
  useEffect(() => {
```

```
    const fetchData = async () => {
```

```
      try {
```

```
        const response = await
```

```
        fetch(`https://cloud.culture.tw/frontsite/trans/SearchShowAction.do?method=doFin
```

```

dTypeJ&category=8`);
        const data = await response.json();
        console.log('Movie Data:', data);
        setMovieData(data); // 將抓回來的資料設定到狀態
movieData 中
        } catch (error) {
            console.log('Error:', error);
        }
    };

    fetchData();
}, [movieData]));

if (!movieData) {
    return <div>Fetching Data...</div>;
}

return (
    <div>
        <h2>movie Data:</h2>
        <p>Title: {movieData[0].title}</p>
        <p>Description: {movieData[0].descriptionFilterHtml}</p>
        { /* 在這裡顯示更多使用者資料 */ }
    </div>
);
};

export default FetchData3;

```

寫一個讀取演唱會程式:

<http://cloud.culture.tw/frontsite/trans/SearchShowAction.do?method=doFindTypeJ&category=17>